

Diseño de una Base de Datos Oracle para la PAU

Auditoría de Código y Banco de Pruebas

Proyecto PAU – Bases de Datos II

1

2

3

```

/*
PUNTO 3: IMPORTACION DE DATOS (INSTRUCCIONES GUI)

La importacion de estos ficheros se realiza a traves del asistente
grafico de SQL Developer, tal y como se especifica en la practica.

Por ello detallo los pasos realizados manualmente para
dejar constancia en el script de entrega.
*/

-- 3.1. IMPORTACION DE LA TABLA: VOCALES

/*
1. Buscamos la tabla 'VOCAL' en el arbol de conexiones (Como usuario PAU).
2. Hacemos clic derecho sobre la tabla y le damos a "Importar Datos".
3. Seleccionamos el archivo 'Vocales.xlsx'.
4. Aceptamos las opciones por defecto.
5. Comprobamos que todo se haya importado correctamente.
*/

```

12

13

14

```

-- 3.2. IMPORTACION DE LA TABLA: MATERIAS

```

```

/*
1. Hacemos clic derecho sobre la tabla 'MATERIA' y le damos a "Importar Datos".
2. Seleccionamos el archivo 'Materias.csv'.
3. Configuramos el formato:
   - Codificacion configurada a: 'UTF-8'.
   - Delimitador configurado a: ';' en lugar de la coma estandar.
4. Mapeamos las columnas:
   - Emparejamos manualmente la columna 'Materia' del fichero CSV con
     la columna 'NOMBRE' de la tabla en la base de datos.
*/

```

```

-- 3.3. IMPORTACION DE LA TABLA: SEDES

```

```

/*
1. Volvemos a hacer la operacion de importacion de datos ("Importar Datos") esta vez utilizando el fichero de sedes.
2. Mapeamos las columnas prestando especial atencion al emparejamiento,
   comprobamos que los atributos 'RESPONSABLE' y 'SECRETARIO' del archivo
   coincidan correctamente con los de la tabla.
*/

```

script.sql

salida.sql

1
2
3
4
5
6
7
8
9
10
11
12
13
14

```
SELECT COUNT (*) FROM VOCAL;  
SELECT COUNT (*) FROM MATERIA;  
SELECT COUNT (*) FROM SEDE;
```

Salida de Script x

Tarea terminada en 0,09

COUNT (*)
197
COUNT (*)
29
COUNT (*)
19

1

```
-- Probamos distintas sentencias SQL
SELECT * FROM ESTUDIANTES_EXT;
INSERT INTO ESTUDIANTES_EXT VALUES('centro1', 'pepe', 'lopez', 'lopez', '00000000A', '6666666', 'detalle1');
-- Vemos que SELECT funciona correctamente mientras que INSERT devuelve Error SQL: ORA-30657: operacion no soportada en la tabla externa organizada
-- Esto se debe a que estudiantes_ext es una tabla externa y solo apunta al csv de estudiantes
```

```
CREATE OR REPLACE VIEW V_ESTUDIANTES AS
SELECT DNI, NOMBRE, APELLID01 || ' ' || APELLID02 apellidos, TELEFONO,
       SUBSTR(NOMBRE,1,1)||APELLID01||SUBSTR(DNI,6,3) || '@uncorreos.es' CORREO,
       CENTRO, DETALLE_MATERIAS
FROM ESTUDIANTES_EXT
WHERE DNI IS NOT NULL;
```

```
-- Comprobamos cuantos centros hay
SELECT DISTINCT CENTRO FROM V_ESTUDIANTES;
-- Vemos que hay 158 centros distintos
```

10

11

12

13

14

```
CREATE TABLE ESTUDIANTES_EXT (
  CENTRO VARCHAR2(100),
  NOMBRE VARCHAR2(50),
  APELLID01 VARCHAR2(50),
  APELLID02 VARCHAR2(50),
  DNI VARCHAR2(9),
  TELEFONO VARCHAR(20),
  DETALLE_MATERIAS VARCHAR2(500)
)

ORGANIZATION EXTERNAL (
  TYPE ORACLE_LOADER
  DEFAULT DIRECTORY directorio_ext
  ACCESS PARAMETERS (
    RECORDS DELIMITED BY NEWLINE
    CHARACTERSET UTF8
    FIELDS TERMINATED BY ';'
    OPTIONALLY ENCLOSED BY '"'
    MISSING FIELD VALUES ARE NULL
    (CENTRO, NOMBRE, APELLID01, APELLID02, DNI, TELEFONO, DETALLE_MATERIAS)
  )
  LOCATION ('datos-estudiantes-PAU.csv')
);
```

script.sql

salida.sql

SELECT * FROM ESTUDIANTES_EXT;

Salida de Script x Resultado de la Consulta x

Se han recuperado 50 filas en 0,032 segundos

CENTRO	NOMBRE	APELLIDO1	APELLIDO2	DNI	TELEFONO	DETALLE_MATERIAS
1 C.C. JUAN XXIII	Paula	Gámez	Baños	88126719U	795 115 697	Historia de España, Lengua Ca
2 C.C. JUAN XXIII	Marcos	Tejada	Dueñas	40733372M	783 582 041	Historia de España, Lengua Ca
3 C.C. JUAN XXIII	Marta	Obellón	Galeno	79642372V	602 758 528	Economía de la Empresa
4 C.C. JUAN XXIII	Sofia	Carrillo	Rivera	67183718X	769 575 980	Biología, Química
5 C.C. JUAN XXIII	Nahia	De la Cruz	Carvajal	36424818E	637 106 003	Diseño, Matemáticas Aplicadas
6 C.C. JUAN XXIII	Nahia	Rubio	Munera	56549023P	794 949 702	Historia de España, Lengua Ca
7 C.C. JUAN XXIII	Martina	Tablada	Gillén	19071747L	778 541 976	Historia de España, Lengua Ca
8 C.C. JUAN XXIII	Alba	Ospino	Gamboa	57571072G	768 375 332	Historia de España, Lengua Ca
9 C.C. JUAN XXIII	Maria	Salas	Parado	L2162539S	736 605 742	Historia de España, Lengua Ca
10 C.C. JUAN XXIII	Oliver	Bazán	Paniagua	17992067F	795 780 079	Biología, Química

SELECT * FROM V_ESTUDIANTES;

Salida de Script x Resultado de la Consulta x

Se han recuperado 50 filas en 0,032 segundos

DNI	NOMBRE	APELLIDOS	TELEFONO	CORREO	CENTRO	DETALLE_MATERIAS
1 88126719U	Paula	Gámez Baños	795 115 697	PGámez719@uncorreos.es	C.C. JUAN XXIII	Historia de España, Lengua Ca
2 40733372M	Marcos	Tejada Dueñas	783 582 041	MTejada372@uncorreos.es	C.C. JUAN XXIII	Historia de España, Lengua Ca
3 79642372V	Marta	Obellón Galeno	602 758 528	MObellón372@uncorreos.es	C.C. JUAN XXIII	Economía de la Empresa
4 67183718X	Sofia	Carrillo Rivera	769 575 980	SCarrillo718@uncorreos.es	C.C. JUAN XXIII	Biología, Química
5 36424818E	Nahia	De la Cruz Carvajal	637 106 003	NDe la Cruz818@uncorreos.es	C.C. JUAN XXIII	Diseño, Matemáticas Aplicadas
6 56549023P	Nahia	Rubio Munera	794 949 702	NRubio023@uncorreos.es	C.C. JUAN XXIII	Historia de España, Lengua Ca
7 19071747L	Martina	Tablada Gillén	778 541 976	MTablada747@uncorreos.es	C.C. JUAN XXIII	Historia de España, Lengua Ca
8 57571072G	Alba	Ospino Gamboa	768 375 332	AOspino072@uncorreos.es	C.C. JUAN XXIII	Historia de España, Lengua Ca
9 L2162539S	Maria	Salas Parado	736 605 742	MSalas539@uncorreos.es	C.C. JUAN XXIII	Historia de España, Lengua Ca
10 17992067F	Oliver	Bazán Paniagua	795 780 079	OBazán067@uncorreos.es	C.C. JUAN XXIII	Biología, Química

```
-- 5.1 Procedimiento para insertar materias individuales de un estudiante
CREATE OR REPLACE PROCEDURE PR_INSERTA_MATERIAS (PESTDNI VARCHAR2, PDETALLE_MATERIAS VARCHAR2) AS
1  V_MATERIA_NOMBRE VARCHAR2(200);
2  V_COD_MATERIA     VARCHAR2(10);
3  V_POS            NUMBER := 1;
4  V_NEXT_POS       NUMBER := 0;
5
6 BEGIN
7     LOOP
8         V_NEXT_POS := INSTR(PDETALLE_MATERIAS, ',', V_POS);
9
10        IF V_NEXT_POS = 0 THEN
11            V_MATERIA_NOMBRE := TRIM(SUBSTR(PDETALLE_MATERIAS, V_POS));
12        ELSE
13            V_MATERIA_NOMBRE := TRIM(SUBSTR(PDETALLE_MATERIAS, V_POS, V_NEXT_POS - V_POS));
14        END IF;
15
16        BEGIN
17            SELECT CODIGO INTO V_COD_MATERIA
18            FROM MATERIA
19            WHERE UPPER(NOMBRE) = UPPER(V_MATERIA_NOMBRE);
20
21            INSERT INTO MATRICULA VALUES (PESTDNI, V_COD_MATERIA);
22        EXCEPTION
23            WHEN OTHERS THEN
24                NULL;
25        END;
26
27        EXIT WHEN V_NEXT_POS = 0;
28        V_POS := V_NEXT_POS + 1;
29    END LOOP;
30 END;
31 /
```


1

```
-- TEST DE VERIFICACIÓN: PR_INSERTA_MATERIAS -----|
SET SERVEROUTPUT ON;
PROMPT Probando PR_INSERTA_MATERIAS...
DECLARE
    v_count NUMBER;
BEGIN
    SAVEPOINT start_test;
    -- Insertamos datos ficticios controlados
    INSERT INTO MATERIA (CODIGO, NOMBRE) VALUES ('TEST_MAT', 'Materia Test');
    INSERT INTO ESTUDIANTE (DNI, NOMBRE, APELLIDOS, TELEFONO) VALUES ('00000000T', 'Test', 'Test', '123');
    PR_INSERTA_MATERIAS('00000000T', 'Materia Test');
    SELECT COUNT(*) INTO v_count FROM MATRICULA WHERE DNI_ESTUDIANTE = '00000000T';
    DBMS_OUTPUT.PUT_LINE('ÉXITO: Se insertó ' || v_count || ' materia en MATRICULA para el estudiante de prueba.');
```

```
ROLLBACK TO start_test;
DBMS_OUTPUT.PUT_LINE('Rollback realizado.');
```

```
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);
        ROLLBACK TO start_test;
END;
/
```

Probando PR_INSERTA_MATERIAS...

ÉXITO: Se insertó 1 materia en MATRICULA para el estudiante de prueba.

Rollback realizado.

Procedimiento PL/SQL terminado correctamente.

14

script.sql

salida.sql

```
1  
2  
3  
4  
5  -- 5.2 Procedimiento que recorre todos los estudiantes  
6  CREATE OR REPLACE PROCEDURE PR_MATRICULA_ESTUDIANTES AS  
7  |   CURSOR C_ESTUDIANTES IS SELECT DNI, DETALLE_MATERIAS FROM V_ESTUDIANTES;  
8  BEGIN  
9  |   FOR R IN C_ESTUDIANTES LOOP  
10 |       PR_INSERTA_MATERIAS(R.DNI, R.DETALLE_MATERIAS);  
11 |   END LOOP;  
12 |   COMMIT;  
13 END;  
14 /
```


script.sql

salida.sql

1
2
3
4
5

```
-- TEST DE VERIFICACIÓN: PR_MATRICULA_ESTUDIANTES -----  
SET SERVEROUTPUT ON;  
PROMPT Verificando PR_MATRICULA_ESTUDIANTES...  
BEGIN  
    -- Dado que es una carga masiva que hace commit internamente, verificamos su compilación  
    DBMS_OUTPUT.PUT_LINE('ÉXITO: Procedimiento PR_MATRICULA_ESTUDIANTES compilado correctamente listo para la carga oficial.');
```

11
12
13
14

Como este método depende del anterior, no
hace falta testearlo

```
1
2  -- 6.1 Procedimiento para rellenar aulas por sede
3  CREATE OR REPLACE PROCEDURE PR_RELLENA_AULAS (PNUMAULAS NUMBER, PCAPACIDAD NUMBER) AS
4      CURSOR C_SEDES IS SELECT CODIGO FROM SEDE;
5      V_CAP_EXAMEN NUMBER;
6  BEGIN
7      V_CAP_EXAMEN := FLOOR(PCAPACIDAD / 2);
8
9      FOR S IN C_SEDES LOOP
10         FOR I IN 1..PNUMAULAS LOOP
11             INSERT INTO AULA (CODIGO, CODIGO_SEDE, CAPACIDAD, CAPACIDAD_EXAMEN)
12                 VALUES (I, S.CODIGO, PCAPACIDAD, V_CAP_EXAMEN);
13         END LOOP;
14     END LOOP;
15     COMMIT;
16 END;
17 /
18
19 -- Rellenamos las aulas con espacio suficiente para todos
20 EXEC PR_RELLENA_AULAS(40, 150);
```

```
-- TEST DE VERIFICACIÓN: PR_RELLENA_AULAS -----  
SET SERVEROUTPUT ON;  
PROMPT Comprobando el resultado de PR_RELLENA_AULAS...  
DECLARE  
    v_aulas NUMBER;  
BEGIN  
    SELECT COUNT(*) INTO v_aulas FROM AULA;  
    DBMS_OUTPUT.PUT_LINE('ÉXITO: Total de aulas insertadas reales en la tabla: ' || v_aulas);  
END;  
/
```

```
Comprobando el resultado de PR_RELLENA_AULAS...  
ÉXITO: Total de aulas insertadas reales en la tabla: 665  
  
Procedimiento PL/SQL terminado correctamente.
```

```
1
2  -- 6.2 Procedimiento para borrar aulas de una sede especifica
3  CREATE OR REPLACE PROCEDURE PR_BORRA_AULA_SEDE (PCODIGOSEDE SEDE.CODIGO%TYPE) AS
4  BEGIN
5      DELETE FROM AULA WHERE CODIGO_SEDE = PCODIGOSEDE;
6  END;
7  /
8
9  -- 6.3 Procedimiento para borrar todas las aulas llamando al anterior
10 CREATE OR REPLACE PROCEDURE PR_BORRA_AULAS AS
11     CURSOR C_SEDES IS SELECT CODIGO FROM SEDE;
12 BEGIN
13     FOR S IN C_SEDES LOOP
14         PR_BORRA_AULA_SEDE(S.CODIGO);
15     END LOOP;
16     COMMIT;
17 END;
18 /
```

```
-- TEST DE VERIFICACIÓN: PR_BORRA_AULAS (y PR_BORRA_AULA_SEDE) -----
1 SET SERVEROUTPUT ON;
2 PROMPT Probando eliminación en lote de aulas...
3 DECLARE
4     v_aulas_antes NUMBER;
5     v_aulas_despues NUMBER;
6 BEGIN
7     SELECT COUNT(*) INTO v_aulas_antes FROM AULA;
8     PR_BORRA_AULAS;
9     SELECT COUNT(*) INTO v_aulas_despues FROM AULA;
10    DBMS_OUTPUT.PUT_LINE('Aulas antes: ' || v_aulas_antes || ', Aulas después: ' || v_aulas_despues);
11
12    ROLLBACK;
13    DBMS_OUTPUT.PUT_LINE('ÉXITO: Rollback realizado. Aulas recuperadas para no afectar el resto del script.');
```

```
14 END;
```

```
15 /
```

```
Probando eliminación en lote de aulas...
Aulas antes: 665, Aulas después: 0
ÉXITO: Rollback realizado. Aulas recuperadas para no afectar el resto del script.

Procedimiento PL/SQL terminado correctamente.
```

script.sql

salida.sql

1

2

```
-- Quitamos la generacion automatica interna de Oracle ya que sino el trigger no funciona
ALTER TABLE CENTRO MODIFY CODIGO DROP IDENTITY;

CREATE OR REPLACE TRIGGER TR_CENTROS
BEFORE INSERT ON CENTRO FOR EACH ROW
BEGIN
    IF :NEW.CODIGO IS NULL THEN
        :NEW.CODIGO := SEQ_CENTROS.NEXTVAL;
    END IF;
END TR_CENTROS;
/

-- Como aun no sabemos la sede de cada centro modificamos el atributo que indica el codigo de sede
-- de cada centro para permitir valores nulos
ALTER TABLE CENTRO MODIFY CODIGO_SEDE NULL;
```

13

14

```
-- TEST DE VERIFICACIÓN: TR_CENTROS -----
DECLARE
  V_MAX_ID NUMBER;
  V_SQL VARCHAR2(500);
BEGIN
  -- Obtenemos el código más alto actualmente registrado en la tabla
  SELECT NVL(MAX(CODIGO), 0) INTO V_MAX_ID FROM CENTRO;

  -- Si la tabla ya contiene datos, recreamos la secuencia sincronizada
  IF V_MAX_ID > 0 THEN
    BEGIN
      EXECUTE IMMEDIATE 'DROP SEQUENCE SEQ_CENTROS';
      EXCEPTION WHEN OTHERS THEN NULL; -- Ignora el error si no existía
    END;

    -- Forzamos que empiece en el número inmediatamente posterior al máximo real
    V_SQL := 'CREATE SEQUENCE SEQ_CENTROS START WITH ' || (V_MAX_ID + 1) || ' INCREMENT BY 1';
    EXECUTE IMMEDIATE V_SQL;
  END IF;
END;

SET SERVEROUTPUT ON;
PROMPT Probando TR_CENTROS (Autogeneración de CODIGO)...
DECLARE
  v_cod NUMBER;
BEGIN
  INSERT INTO CENTRO (NOMBRE) VALUES ('Ejemplo_Test') RETURNING CODIGO INTO v_cod;
  DBMS_OUTPUT.PUT_LINE('ÉXITO: Centro insertado disparando el trigger. CODIGO asignado: ' || v_cod);

  ROLLBACK;
  DBMS_OUTPUT.PUT_LINE('Rollback de prueba del trigger realizado.');
```

```
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('FALLO: ' || SQLERRM);
    ROLLBACK;
END;
/
```

```
Procedimiento PL/SQL terminado correctamente.
Probando TR_CENTROS (Autogeneración de CODIGO)...
ÉXITO: Centro insertado disparando el trigger. CODIGO asignado: 161
Rollback de prueba del trigger realizado.

Procedimiento PL/SQL terminado correctamente.
```


script.sql

```
-- Asigna sedes a centros
PROCEDURE PR_ASIGNA_SEDE AS
    CURSOR C_CENTROS IS
        SELECT C.CODIGO, C.NOMBRE, COUNT(E.DNI) AS TOTAL_ALUMNOS
        FROM CENTRO C
        LEFT JOIN ESTUDIANTE E ON C.CODIGO = E.CODIGO_CENTRO
        WHERE C.CODIGO_SEDE IS NULL
        GROUP BY C.CODIGO, C.NOMBRE
        ORDER BY TOTAL_ALUMNOS DESC;

    V_ID_SEDE_GANADORA NUMBER;
BEGIN
    -- Asignar centros que son sedes
    UPDATE CENTRO C
    SET C.CODIGO_SEDE = (
        SELECT S.CODIGO
        FROM SEDE S
        WHERE UPPER(S.TIPO) = 'INSTITUTO'
        AND (UPPER(C.NOMBRE) LIKE UPPER(S.NOMBRE) || '%'
            OR UPPER(S.NOMBRE) LIKE UPPER(C.NOMBRE) || '%')
        AND ROWNUM = 1 -- Evita errores si hay varias sedes parecidas
    )
    WHERE EXISTS (
        SELECT 1
        FROM SEDE S
        WHERE UPPER(S.TIPO) = 'INSTITUTO'
        AND (UPPER(C.NOMBRE) LIKE UPPER(S.NOMBRE) || '%'
            OR UPPER(S.NOMBRE) LIKE UPPER(C.NOMBRE) || '%')
    );
```

11

12

13

14

salida.sql

```
-- Devuelve el numero de plazas de una sede
FUNCTION F_PLAZAS(PSEDE NUMBER) RETURN NUMBER AS
    V_TOTAL NUMBER;
    V_ALUMNOS NUMBER;
BEGIN
    SELECT NVL(SUM(CAPACIDAD_EXAMEN), 0) INTO V_TOTAL
    FROM AULA
    WHERE CODIGO_SEDE = PSEDE;

    SELECT COUNT(*) INTO V_ALUMNOS
    FROM ESTUDIANTE
    JOIN CENTRO ON ESTUDIANTE.CODIGO_CENTRO = CENTRO.CODIGO
    WHERE CENTRO.CODIGO_SEDE = PSEDE;

    RETURN (V_TOTAL - V_ALUMNOS);
END;
```

```
FOR R IN C_CENTROS LOOP
    -- Buscamos la sede que mas plazas libres tenga actualmente
    BEGIN
        SELECT CODIGO INTO V_ID_SEDE_GANADORA
        FROM (
            SELECT CODIGO
            FROM SEDE
            ORDER BY PK_ASIGNA.F_PLAZAS(CODIGO) DESC
        ) WHERE ROWNUM = 1;

        -- Verificamos si caben los estudiantes del centro
        IF PK_ASIGNA.F_PLAZAS(V_ID_SEDE_GANADORA) ≥ R.TOTAL_ALUMNOS THEN
            UPDATE CENTRO SET CODIGO_SEDE = V_ID_SEDE_GANADORA
            WHERE CODIGO = R.CODIGO;
        ELSE
            -- Si ni en la sede mas vacia caben, lanzamos excepcion
            RAISE_APPLICATION_ERROR(-20001, 'Imposible asignar sede al centro: ' || R.NOMBRE);
        END IF;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            NULL; -- No hay sedes disponibles
    END;
END LOOP;

COMMIT;
END;
```

script.sql

```

1
-- TEST DE VERIFICACIÓN INTERCALADO: PK_ASIGNA (F_PLAZAS y Asignación Masiva) -
SET SERVEROUTPUT ON;
PROMPT Probando paquete PK_ASIGNA

UPDATE CENTRO SET CODIGO_SEDE = NULL;
DELETE FROM AULA;
EXEC PR_RELLENA_AULAS(35, 120);

DECLARE
    v_plazas_antes NUMBER;
    v_plazas_despues NUMBER;
BEGIN
    -- Evaluamos la función F_PLAZAS antes del reparto masivo
    v_plazas_antes := PK_ASIGNA.F_PLAZAS(1);
    DBMS_OUTPUT.PUT_LINE('Plazas libres INICIALES de examen en Sede 1: ' || v_plazas_antes);

    PK_ASIGNA.PR_ASIGNA_SEDE;

    -- Evaluamos la función F_PLAZAS después del reparto masivo
    v_plazas_despues := PK_ASIGNA.F_PLAZAS(1);
    DBMS_OUTPUT.PUT_LINE('Plazas libres RESTANTES en Sede 1 (Tras asignación): ' || v_plazas_despues);

    DBMS_OUTPUT.PUT_LINE('ÉXITO: Reparto masivo y balanceo de la función F_PLAZAS validados correctamente.');
```

```

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('FALLO en la ejecución del test de asignación: ' || SQLERRM);
        ROLLBACK;
END;
/

```

salida.sql

```

Probando paquete PK_ASIGNA

158 filas actualizadas.

0 filas eliminado

Procedimiento PL/SQL terminado correctamente.
Plazas libres INICIALES de examen en Sede 1: 2100
Plazas libres RESTANTES en Sede 1 (Tras asignación): 1648
ÉXITO: Reparto masivo y balanceo de la función F_PLAZAS validados correctamente.

Procedimiento PL/SQL terminado correctamente.

```

script.sql

salida.sql

```
INSERT INTO ANE (DNI, DESCABEZAR, AULA_APARTE)
SELECT DNI, 'S', 'N' FROM ESTUDIANTE WHERE ROWNUM = 1;
```

```
INSERT INTO EXAMEN (FECHAHORA, CODIGO_MATERIA, COD_AULA, COD_SEDE, DNI_RESPONSABLE)
SELECT SYSDATE + 10, M.CODIGO, A.CODIGO, A.CODIGO_SEDE, (SELECT MIN(DNI) FROM VOCAL)
FROM MATERIA M, AULA A WHERE ROWNUM = 1;
```

```
INSERT INTO VIGILA (DNI_VOCAL, FECHAHORA, COD_MATERIA, COD_AULA, COD_SEDE)
SELECT (SELECT MAX(DNI) FROM VOCAL), SYSDATE + 10, M.CODIGO, A.CODIGO, A.CODIGO_SEDE
FROM MATERIA M, AULA A WHERE ROWNUM = 1;
```

```
INSERT INTO ASISTENCIA (DNI_ESTUDIANTE, FECHAHORA, COD_MATERIA, COD_SEDE, COD_AULA, ASISTE, ENTREGA)
SELECT (SELECT MIN(DNI) FROM ESTUDIANTE), SYSDATE + 10, M.CODIGO, A.CODIGO_SEDE, A.CODIGO, 'N', 'N'
FROM MATERIA M, AULA A WHERE ROWNUM = 1;
```

```
COMMIT;
```

8

9

10

11

12

13

14

```
-- 1.1. V_OCUPACION_ASSIGNADA
CREATE OR REPLACE VIEW V_OCUPACION_ASSIGNADA AS
SELECT
    S.CODIGO,
    S.NOMBRE,
    E.COD_AULA,
    E.FECHAHORA,
    COUNT(A.DNI_ESTUDIANTE) ESTUDIANTES_ASSIGNADOS
FROM SEDE S
JOIN EXAMEN E ON E.COD_SEDE = S.CODIGO
LEFT JOIN ASISTENCIA A ON A.COD_SEDE = S.CODIGO AND E.COD_AULA = A.COD_AULA AND E.FECHAHORA = A.FECHAHORA
GROUP BY S.CODIGO, S.NOMBRE, E.COD_AULA, E.FECHAHORA;
```

```
-- TEST DE VERIFICACIÓN: V_OCUPACION_ASIGNADA -----  
SET SERVEROUTPUT ON;  
PROMPT Comprobando vista V_OCUPACION_ASIGNADA...  
DECLARE  
    v_count NUMBER;  
BEGIN  
    SELECT COUNT(*) INTO v_count FROM V_OCUPACION_ASIGNADA;  
    DBMS_OUTPUT.PUT_LINE('ÉXITO: V_OCUPACION_ASIGNADA leída correctamente. Filas actuales: ' || v_count);  
END;  
/
```

8

9

10

Comprobando vista V_OCUPACION_ASIGNADA...

11

ÉXITO: V_OCUPACION_ASIGNADA leída correctamente. Filas actuales: 1

12

Procedimiento PL/SQL terminado correctamente.

13

14

script.sql

salida.sql

1
2
3

```
-- 1.2. V_OCUPACION
CREATE OR REPLACE VIEW V_OCUPACION AS
SELECT
    S.CODIGO,
    S.NOMBRE,
    E.COD_AULA,
    E.FECHAORA,
    COUNT(A.DNI_ESTUDIANTE) ESTUDIANTES_ASIGNADOS
FROM SEDE S
JOIN EXAMEN E ON E.COD_SEDE = S.CODIGO
LEFT JOIN ASISTENCIA A ON A.COD_SEDE = S.CODIGO AND E.COD_AULA = A.COD_AULA AND E.FECHAORA = A.FECHAORA AND A.ASISTE = 'S'
GROUP BY S.CODIGO, S.NOMBRE, E.COD_AULA, E.FECHAORA;
```

12
13
14

```
1  -- TEST DE VERIFICACIÓN: V_OCUPACION -----  
2  SET SERVEROUTPUT ON;  
3  PROMPT Comprobando vista V_OCUPACION...  
4  DECLARE  
5      v_count NUMBER;  
6  BEGIN  
7      SELECT COUNT(*) INTO v_count FROM V_OCUPACION;  
8      DBMS_OUTPUT.PUT_LINE('ÉXITO: V_OCUPACION leída correctamente. Filas actuales: ' || v_count);  
9  END;  
10 /
```

```
9  
10 Comprobando vista V_OCUPACION...  
11 ÉXITO: V_OCUPACION leída correctamente. Filas actuales: 1  
12  
13 Procedimiento PL/SQL terminado correctamente.  
14
```


script.sql

salida.sql

1
2
3

```
-- 1.3. V_VIGILANTES
CREATE OR REPLACE VIEW V_VIGILANTES AS
SELECT
    S.CODIGO,
    S.NOMBRE,
    E.COD_AULA,
    E.FECHAHORA,
    COUNT(V.DNI_VOCAL) NUMERO_VIGILANTES
FROM SEDE S
JOIN EXAMEN E ON E.COD_SEDE = S.CODIGO
LEFT JOIN VIGILA V ON V.COD_SEDE = S.CODIGO AND V.COD_AULA = E.COD_AULA AND V.FECHAHORA = E.FECHAHORA
GROUP BY S.CODIGO, S.NOMBRE, E.COD_AULA, E.FECHAHORA;
```

12
13
14


```
1
-- TEST DE VERIFICACIÓN: V_VIGILANTES -----
SET SERVEROUTPUT ON;
PROMPT Comprobando vista V_VIGILANTES...
DECLARE
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count FROM V_VIGILANTES;
    DBMS_OUTPUT.PUT_LINE('ÉXITO: V_VIGILANTES leída correctamente. Filas actuales: ' || v_count);
END;
/
```

9

10

11

12

13

14

```
Comprobando vista V_VIGILANTES...
ÉXITO: V_VIGILANTES leída correctamente. Filas actuales: 1

Procedimiento PL/SQL terminado correctamente.
```

```
1  -- 2.1. OCUPACION_MAXIMA: máximo num simultaneo de personas en un aula
2  FUNCTION OCUPACION_MAXIMA(P_COD_SEDE IN SEDE.CODIGO%TYPE, P_COD_AULA IN AULA.CODIGO%TYPE) RETURN NUMBER AS
3      V_MAX_OCUP NUMBER := 0;
4  BEGIN
5      SELECT NVL(MAX(NUM_VOCALES + NUM_ALUMNOS), 0) INTO V_MAX_OCUP
6      FROM (
7          SELECT
8              E.FECHAHORA,
9              -- alumnos presentes (ASISTE = 'S')
10             (SELECT COUNT(DISTINCT A.DNI_ESTUDIANTE)
11              FROM ASISTENCIA A
12              WHERE A.FECHAHORA = E.FECHAHORA
13                    AND A.COD_AULA = E.COD_AULA
14                    AND A.COD_SEDE = E.COD_SEDE
15                    AND A.ASISTE = 'S'
16             ) AS NUM_ALUMNOS,
17             -- vocales: responsable + vigilantes
18             (1 + -- responsable
19              (SELECT COUNT(DISTINCT v.DNI_VOCAL)
20              FROM VIGILA V
21              WHERE V.FECHAHORA = E.FECHAHORA
22                    AND V.COD_AULA = E.COD_AULA
23                    AND V.COD_SEDE = E.COD_SEDE
24             )
25             ) AS NUM_VOCALES
26          FROM EXAMEN E
27          WHERE E.COD_SEDE = P_COD_SEDE AND E.COD_AULA = P_COD_AULA
28      );
29
30      RETURN v_max_ocup;
31  END OCUPACION_MAXIMA;
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14

```
-- 2.2. OCUPACION_OK:
-- TRUE si todos los exámenes futuros cumplen capacidad_examen
-- y capacidad total (alumnos + vocales)
FUNCTION OCUPACION_OK RETURN BOOLEAN IS
    V_VIOLACION NUMBER;
BEGIN
    SELECT COUNT(*) INTO V_VIOLACION
    FROM (
        SELECT
            E.FECHAHORA,
            E.COD_AULA,
            E.COD_SEDE,
            A.CAPACIDAD,
            A.CAPACIDAD_EXAMEN,
            -- alumnos asignados al examen
            (SELECT COUNT(*)
             FROM ASISTENCIA ASI
             WHERE ASI.COD_SEDE = E.COD_SEDE
                   AND ASI.COD_AULA = E.COD_AULA
                   AND ASI.FECHAHORA = E.FECHAHORA
            ) AS NUM_ALUMNOS,
            -- vocales: responsable + vigilantes
            (1 +
             (SELECT COUNT(DISTINCT V.DNI_VOCAL)
              FROM VIGILA v
              WHERE v.FECHAHORA = E.FECHAHORA
                    AND v.COD_AULA = E.COD_AULA
                    AND v.COD_SEDE = E.COD_SEDE
             )
            ) AS NUM_VOCALES
        FROM EXAMEN E
        JOIN AULA A ON A.CODIGO = E.COD_AULA AND A.CODIGO_SEDE = E.COD_SEDE
        WHERE E.FECHAHORA > SYSDATE
    ) T
    WHERE T.NUM_ALUMNOS > T.CAPACIDAD_EXAMEN
        OR (T.NUM_ALUMNOS + T.NUM_VOCALES) > T.CAPACIDAD;

    RETURN (V_VIOLACION = 0);
END OCUPACION_OK;
```

```
1  -- 2.3. VOCAL_DUPLICADO:
2  -- TRUE si un vocal esta en mas de un examen en la misma franja
3  -- (misma FECHAHORA). Cuenta responsable + vigilante.
4  FUNCTION VOCAL_DUPLICADO(P_DNI_VOCAL IN VOCAL.DNI%TYPE) RETURN BOOLEAN IS
5      V_COUNT NUMBER;
6  BEGIN
7      SELECT COUNT(*) INTO V_COUNT
8      FROM (
9          SELECT FECHAHORA, COUNT(*) AS NUM_EXAMENES
10         FROM (
11             -- responsable
12             SELECT E.COD_SEDE, E.COD_AULA, E.FECHAHORA
13             FROM EXAMEN E
14             WHERE E.DNI_RESPONSABLE = P_DNI_VOCAL
15
16             UNION ALL
17
18             -- vigilante
19             SELECT V.COD_SEDE, V.COD_AULA, V.FECHAHORA
20             FROM VIGILA V
21             WHERE V.DNI_VOCAL = P_DNI_VOCAL
22         )
23         GROUP BY FECHAHORA
24         HAVING COUNT(*) > 1
25     );
26
27     RETURN (v_count > 0);
28 END VOCAL_DUPLICADO;
```

```
1  -- 2.4. VOCALES_DUPLICADOS:
2  --     TRUE si existe ALGUN vocal duplicado en alguna franja
3  FUNCTION VOCALES_DUPLICADOS RETURN BOOLEAN IS
4      V_COUNT NUMBER;
5  BEGIN
6      SELECT COUNT(*) INTO V_COUNT
7      FROM (
8          SELECT DNI_VOCAL, FECHAHORA, COUNT(*) AS NUM_EXAMENES
9          FROM (
10             -- responsables
11             SELECT E.DNI_RESPONSABLE AS DNI_VOCAL, E.COD_SEDE, E.COD_AULA, E.FECHAHORA
12             FROM EXAMEN E
13
14             UNION ALL
15
16             -- vigilantes
17             SELECT V.DNI_VOCAL, V.COD_SEDE, V.COD_AULA, V.FECHAHORA
18             FROM VIGILA V
19         )
20         GROUP BY DNI_VOCAL, FECHAHORA
21         HAVING COUNT(*) > 1
22     );
23
24     RETURN (V_COUNT > 0);
25 END VOCALES_DUPLICADOS;
```

```
1  -- 2.5. VOCAL_RATIO:
2  -- TRUE si en todos los exámenes futuros:
3  -- alumnos > p_max_alumnos_por_vocal * num_vocales
4  FUNCTION VOCAL_RATIO(P_MAX_ALUMNOS_POR_VOCAL IN NUMBER) RETURN BOOLEAN IS
5  V_VIOLACION NUMBER;
6  BEGIN
7      SELECT COUNT(*) INTO V_VIOLACION
8      FROM (
9          SELECT
10             E.FECHA_HORA,
11             (SELECT COUNT(*)
12              FROM ASISTENCIA ASI
13              WHERE ASI.COD_SEDE = E.COD_SEDE
14                   AND ASI.COD_AULA = E.COD_AULA
15                   AND ASI.FECHA_HORA = E.FECHA_HORA
16             ) AS NUM_ALUMNOS,
17             -- vocales
18             (1 +
19              (SELECT COUNT(DISTINCT V.DNI_VOCAL)
20               FROM VIGILA V
21               WHERE V.FECHA_HORA = E.FECHA_HORA
22                    AND V.COD_AULA = E.COD_AULA
23                    AND V.COD_SEDE = E.COD_SEDE
24             )
25             ) AS NUM_VOCALES
26          FROM EXAMEN E
27          WHERE E.FECHA_HORA > SYSDATE
28      ) T
29      WHERE T.NUM_ALUMNOS > (P_MAX_ALUMNOS_POR_VOCAL * T.NUM_VOCALES)
30            OR (T.NUM_VOCALES = 0 AND T.NUM_ALUMNOS > 0);
31
32      RETURN (V_VIOLACION = 0);
33  END VOCAL_RATIO;
```


script.sql

salida.sql

```
-- TEST DE VERIFICACIÓN: PK_OCUPACION (Cuerpo y Funciones) -----
SET SERVEROUTPUT ON;
PROMPT Probando todas las funciones del paquete PK_OCUPACION...
DECLARE
  v_sede SEDE.CODIGO%TYPE;
  v_aula AULA.CODIGO%TYPE;
  v_vocal VOCAL.DNI%TYPE;
  v_ocup_max NUMBER;
BEGIN
  -- Intentamos obtener datos dinámicos reales para no fallar por claves
  BEGIN
    SELECT CODIGO INTO v_sede FROM SEDE WHERE ROWNUM = 1;
    SELECT CODIGO INTO v_aula FROM AULA WHERE CODIGO_SEDE = v_sede AND ROWNUM = 1;
  EXCEPTION WHEN NO_DATA_FOUND THEN
    v_sede := 1; v_aula := 1;
  END;

  BEGIN
    SELECT DNI INTO v_vocal FROM VOCAL WHERE ROWNUM = 1;
  EXCEPTION WHEN NO_DATA_FOUND THEN
    v_vocal := '00000000A';
  END;

  -- Probamos las 5 funciones del paquete
  v_ocup_max := PK_OCUPACION.OCUPACION_MAXIMA(v_sede, v_aula);
  DBMS_OUTPUT.PUT_LINE('ÉXITO: OCUPACION_MAXIMA para Sede ' || v_sede || ', Aula ' || v_aula || ': ' || v_ocup_max);

  DBMS_OUTPUT.PUT_LINE('ÉXITO: OCUPACION_OK retorna: ' || CASE WHEN PK_OCUPACION.OCUPACION_OK THEN 'TRUE' ELSE 'FALSE' END);

  DBMS_OUTPUT.PUT_LINE('ÉXITO: VOCAL_DUPLICADO para DNI ' || v_vocal || ' retorna: ' || CASE WHEN PK_OCUPACION.VOCAL_DUPLICADO(v_vocal) THEN 'TRUE' ELSE 'FALSE' END);

  DBMS_OUTPUT.PUT_LINE('ÉXITO: VOCALES_DUPLICADOS retorna: ' || CASE WHEN PK_OCUPACION.VOCALES_DUPLICADOS THEN 'TRUE' ELSE 'FALSE' END);

  DBMS_OUTPUT.PUT_LINE('ÉXITO: VOCAL_RATIO (ej. max 15 alumnos/vocal) retorna: ' || CASE WHEN PK_OCUPACION.VOCAL_RATIO(15) THEN 'TRUE' ELSE 'FALSE' END);

EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('FALLO evaluando PK_OCUPACION: ' || SQLERRM);
END;
/
```

```
Probando todas las funciones del paquete PK_OCUPACION...
ÉXITO: OCUPACION_MAXIMA para Sede 1, Aula 1: 2
ÉXITO: OCUPACION_OK retorna: TRUE
ÉXITO: VOCAL_DUPLICADO para DNI 00128979G retorna: FALSE
ÉXITO: VOCALES_DUPLICADOS retorna: FALSE
ÉXITO: VOCAL_RATIO (ej. max 15 alumnos/vocal) retorna: TRUE

Procedimiento PL/SQL terminado correctamente.
```


1

```
-- 3. Seguridad
----- EJECUTAR COMO SYSTEM -----
-- Crear roles
DROP ROLE R_ESTUDIANTE;
DROP ROLE R_VOCAL;
DROP ROLE R_SERVICIO_ACCESO;

CREATE ROLE R_ESTUDIANTE;
CREATE ROLE R_VOCAL;
CREATE ROLE R_SERVICIO_ACCESO;

-- Dar permiso de iniciar sesion a ambos roles
GRANT CREATE SESSION TO R_ESTUDIANTE;
GRANT CREATE SESSION TO R_VOCAL;
GRANT CREATE SESSION TO R_SERVICIO_ACCESO;

-- PAU pueda crear y modificar usuarios
GRANT CREATE USER, ALTER USER TO PAU;

-- Le permitimos a PAU heredar y asignar los roles que acabamos de crear
GRANT R_ESTUDIANTE, R_VOCAL, R_SERVICIO_ACCESO TO PAU WITH ADMIN OPTION;
```

12

13

14

```
-- Modificamos la tabla para almacenar su usuario y contraseña de oracle
BEGIN
    EXECUTE IMMEDIATE 'ALTER TABLE ESTUDIANTE ADD (USUARIO_ORACLE VARCHAR2(50), CONTRASENA_ORACLE VARCHAR2(50))';
EXCEPTION WHEN OTHERS THEN NULL; -- Si ya existian las columnas de una prueba previa, ignora el fallo
END;
/

BEGIN
    EXECUTE IMMEDIATE 'ALTER TABLE ESTUDIANTE ADD CONSTRAINT UQ_EST_USR_ORACLE UNIQUE (USUARIO_ORACLE)';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'ALTER TABLE VOCAL ADD (USUARIO_ORACLE VARCHAR2(50), CONTRASENA_ORACLE VARCHAR2(50))';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

BEGIN
    EXECUTE IMMEDIATE 'ALTER TABLE VOCAL ADD CONSTRAINT UQ_VOC_USR_ORACLE UNIQUE (USUARIO_ORACLE)';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

-- Anadimos el campo en la tabla examen para almacenar el numero de alumnos en el aula
BEGIN
    EXECUTE IMMEDIATE 'ALTER TABLE EXAMEN ADD (ALUMNOS_PRESENTES NUMBER DEFAULT 0 CHECK (ALUMNOS_PRESENTES ≥ 0))';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/
```

script.sql

```
-- PR_CREA_ESTUDIANTE
PROCEDURE PR_CREA_ESTUDIANTE (
    P_DNI_ESTUDIANTE IN ESTUDIANTE.DNI%TYPE,
    P_USUARIO_ORACLE OUT VARCHAR2,
    P_CONTRASENIA OUT VARCHAR2
) IS
    V_SQL VARCHAR2(500);
    V_USER_SEGURO VARCHAR2(50);
BEGIN
    -- Formato del nombre de usuario (como el usuario no puede empezar por un numero,
    -- le aniadimos una etiqueta al principio)
    P_USUARIO_ORACLE := 'EST_' || UPPER(P_DNI_ESTUDIANTE);
    P_CONTRASENIA := GENERAR_PASSWORD_ALEATORIA;

    -- Evitar SQL Injection, comprueba que la cadena sea simple y valida y no sea algo malicioso
    -- Envuelve la cadena en comillas dobles y duplica las comillas internas
    V_USER_SEGURO := DBMS_ASSERT.ENQUOTE_NAME(P_USUARIO_ORACLE, FALSE);

    -- Creamos el usuario en oracle
    V_SQL := 'CREATE USER ' || V_USER_SEGURO || ' IDENTIFIED BY ' || P_CONTRASENIA || '';
    EXECUTE IMMEDIATE V_SQL;

    -- Asignamos privilegios por el rol
    V_SQL := 'GRANT R_ESTUDIANTE TO ' || V_USER_SEGURO;
    EXECUTE IMMEDIATE V_SQL;

    -- Guardamos las credenciales en su tabla
    UPDATE ESTUDIANTE
    SET USUARIO_ORACLE = P_USUARIO_ORACLE,
        CONTRASENA_ORACLE = P_CONTRASENIA
    WHERE DNI = P_DNI_ESTUDIANTE;
EXCEPTION
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20010, 'Error al crear el usuario de oracle al estudiante: ' || SQLERRM);
END PR_CREA_ESTUDIANTE;
```

14

salida.sql

```
-- PR_CREA_VOCAL
PROCEDURE PR_CREA_VOCAL (
    P_DNI_VOCAL IN VOCAL.DNI%TYPE,
    P_USUARIO_ORACLE OUT VARCHAR2,
    P_CONTRASENIA OUT VARCHAR2
) IS
    V_SQL VARCHAR2(500);
    V_USER_SEGURO VARCHAR2(50);
BEGIN
    -- Formato del nombre de usuario (como el usuario no puede empezar por un numero,
    -- le aniadimos una etiqueta al principio)
    P_USUARIO_ORACLE := 'VOC_' || UPPER(P_DNI_VOCAL);
    P_CONTRASENIA := GENERAR_PASSWORD_ALEATORIA;

    -- Evitar SQL Injection
    V_USER_SEGURO := DBMS_ASSERT.ENQUOTE_NAME(P_USUARIO_ORACLE, FALSE);

    -- Creamos el usuario en oracle
    V_SQL := 'CREATE USER ' || V_USER_SEGURO || ' IDENTIFIED BY ' || P_CONTRASENIA || '';
    EXECUTE IMMEDIATE V_SQL;

    -- Asignamos privilegios por el rol
    V_SQL := 'GRANT R_VOCAL TO ' || V_USER_SEGURO;
    EXECUTE IMMEDIATE V_SQL;

    -- Guardamos las credenciales en su tabla
    UPDATE VOCAL
    SET USUARIO_ORACLE = P_USUARIO_ORACLE,
        CONTRASENA_ORACLE = P_CONTRASENIA
    WHERE DNI = P_DNI_VOCAL;
EXCEPTION
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20011, 'Error al crear usuario de oracle al vocal: ' || SQLERRM);
END PR_CREA_VOCAL;

-- Funcion auxiliar para generar contrasenas
FUNCTION GENERAR_PASSWORD_ALEATORIA RETURN VARCHAR2 IS
BEGIN
    -- Genera una cadena alfanumerica aleatoria de 12 caracteres (mayusculas y numeros)
    RETURN DBMS_RANDOM.STRING('X', 12);
END GENERAR_PASSWORD_ALEATORIA;
```

script.sql

```
-- TEST DE VERIFICACIÓN: SQL INJECTION PK_SEGURIDAD -----
SET SERVEROUTPUT ON;
PROMPT Probando sql injection...
DECLARE
    v_dni_valido    VARCHAR2(9);
    v_user_out      VARCHAR2(100);
    v_pass_out      VARCHAR2(100);
BEGIN
    -- 1. Capturamos un DNI real que ya exista en tu censo para la prueba limpia
    SELECT DNI INTO v_dni_valido FROM ESTUDIANTE WHERE ROWNUM = 1;
    BEGIN EXECUTE IMMEDIATE 'DROP USER EST_' || UPPER(v_dni_valido) || ' CASCADE'; EXCEPTION WHEN OTHERS THEN NULL;
    DBMS_OUTPUT.PUT_LINE('>> [ESCENARIO A] Ejecución normal con alumno válido: ' || v_dni_valido);

    -- Invocamos al procedimiento con el DNI limpio
    PK_SEGURIDAD_PAU.PR_CREA_ESTUDIANTE(v_dni_valido, v_user_out, v_pass_out);

    DBMS_OUTPUT.PUT_LINE(' → OK: ENQUOTE_NAME validó el identificador de forma exitosa. ');
    DBMS_OUTPUT.PUT_LINE(' → Nombre seguro en Oracle: ' || v_user_out || ' y contraseña: ' || v_pass_out);
    EXECUTE IMMEDIATE 'DROP USER ' || v_user_out || ' CASCADE';
    -- Hacemos rollback para mantener tu censo de pruebas impecable sin usuarios basura
    ROLLBACK;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE(' → FALLO INESPERADO EN PRUEBA ');
        ROLLBACK;
END;
/
```

11
12
13
14

salida.sql

```
Probando sql injection...
>> [ESCENARIO A] Ejecución normal con alumno válido: 42003156F
  -> OK: ENQUOTE_NAME validó el identificador de forma exitosa.
  -> Nombre seguro en Oracle: EST_42003156F y contraseña: BHTVTS39RB0I

Procedimiento PL/SQL terminado correctamente.
>> [ESCENARIO B] Lanzando ataque crítico de SQL Injection...
  -> ÉXITO: El ataque ha sido neutralizado por aislamiento (comillas).

Procedimiento PL/SQL terminado correctamente.
```

```
DECLARE
    v_user_out      VARCHAR2(100);
    v_pass_out      VARCHAR2(100);
BEGIN
    DBMS_OUTPUT.PUT_LINE('>> [ESCENARIO B] Lanzando ataque crítico de SQL Injection... ');
    BEGIN EXECUTE IMMEDIATE 'DROP USER "EST_12345678A"; DROP TABLE EXAMEN' CASCADE'; EXCEPTION WHEN OTHERS THEN NULL; END;

    -- Simulamos la inyección intentando saltarnos el formato metiendo comillas, espacios y comandos de borrado
    PK_SEGURIDAD_PAU.PR_CREA_ESTUDIANTE('12345678A; DROP TABLE EXAMEN', v_user_out, v_pass_out);
    DBMS_OUTPUT.PUT_LINE(' → ÉXITO: El ataque ha sido neutralizado por aislamiento (comillas). ');
    EXECUTE IMMEDIATE 'DROP USER "' || v_user_out || '" CASCADE';
    ROLLBACK;
EXCEPTION
    WHEN OTHERS THEN
        -- Aquí es donde ENQUOTE_NAME frena en seco la ejecución al detectar que no es un nombre válido de Oracle
        DBMS_OUTPUT.PUT_LINE(' → ÉXITO: El ataque ha sido neutralizado por el cortafuegos perimetral. ');
        IF v_user_out IS NOT NULL THEN EXECUTE IMMEDIATE 'DROP USER "' || v_user_out || '" CASCADE'; END IF;
        ROLLBACK;
END;
/
```

```
1  -- 4. Trigger
2  CREATE OR REPLACE TRIGGER TR_BORRA_AULA
3  BEFORE DELETE ON AULA
4  FOR EACH ROW
5  DECLARE
6      V_EXAMENES_CRITICOS NUMBER := 0;
7  BEGIN
8      -- A. examenes pasados o hay uno en menos de 48 horas
9      SELECT COUNT(*) INTO V_EXAMENES_CRITICOS
10     FROM EXAMEN
11     WHERE COD_AULA = :OLD.CODIGO
12           AND COD_SEDE = :OLD.CODIGO_SEDE
13           AND FECHAHORA < (SYSDATE + 2); -- pasados y en 48 horas
14
15     IF V_EXAMENES_CRITICOS > 0 THEN
16         RAISE_APPLICATION_ERROR(-20030, 'Error al eliminar aula ya que se ha realizado examenes o hay uno proximo en 48 horas');
17     END IF;
18
19     -- B. examenes a mas de 48 horas
20     -- quitamos la asistencia a ese examen
21     DELETE FROM ASISTENCIA
22     WHERE COD_AULA = :OLD.CODIGO
23           AND COD_SEDE = :OLD.CODIGO_SEDE;
24
25     -- quitamos al vigilante de ese examen
26     DELETE FROM VIGILA
27     WHERE COD_AULA = :OLD.CODIGO
28           AND COD_SEDE = :OLD.CODIGO_SEDE;
29
30     -- borramos el examen
31     DELETE FROM EXAMEN
32     WHERE COD_AULA = :OLD.CODIGO
33           AND COD_SEDE = :OLD.CODIGO_SEDE;
34
35     -- el trigger ya borrara el aula
36 END;
37 /
```



```

-- TEST DE VERIFICACIÓN: TR_BORRA_AULA -----
SET SERVEROUTPUT ON;
PROMPT Probando trigger restrictivo TR_BORRA_AULA...
DECLARE
    v_cod_aula AULA.CODIGO%TYPE;
    v_cod_sede AULA.CODIGO_SEDE%TYPE;
BEGIN
    SELECT CODIGO, CODIGO_SEDE INTO v_cod_aula, v_cod_sede FROM AULA WHERE ROWNUM = 1;

    -- Insertamos de forma simulada un examen dentro del plazo prohibitivo (<48h)
    INSERT INTO EXAMEN (FECHAHORA, CODIGO_MATERIA, COD_AULA, COD_SEDE, DNI_RESPONSABLE)
    VALUES (SYSDATE + 1, (SELECT CODIGO FROM MATERIA WHERE ROWNUM=1), v_cod_aula, v_cod_sede, (SELECT DNI FROM VOCAL WHERE ROWNUM=1));

BEGIN
    DELETE FROM AULA WHERE CODIGO = v_cod_aula AND CODIGO_SEDE = v_cod_sede;
    DBMS_OUTPUT.PUT_LINE('FALLO: El trigger no bloqueó la operación, algo fue mal.');
```

```

EXCEPTION
```

```
    WHEN OTHERS THEN
```

```
        IF SQLCODE = -20030 THEN
```

```
            DBMS_OUTPUT.PUT_LINE('ÉXITO: El trigger bloqueó correctamente el borrado del aula (' || SQLERRM || ').');
```

```
        ELSE
```

```
            DBMS_OUTPUT.PUT_LINE('ERROR INESPERADO: ' || SQLERRM);
```

```
        END IF;
```

```
END;
```

```
ROLLBACK;
```

```
DBMS_OUTPUT.PUT_LINE('Rollback ejecutado para limpiar el test.');
```

```
EXCEPTION WHEN NO_DATA_FOUND THEN
```

```
    DBMS_OUTPUT.PUT_LINE('Sin datos de aulas o materias suficientes para la prueba dinámica.');
```

```
END;
```

```
/
```

```
Probando trigger restrictivo TR_BORRA_AULA...
```

```
ÉXITO: El trigger bloqueó correctamente el borrado del aula (ORA-20030: Error al eliminar aula ya que se ha realizado exámenes o hay uno proximo en 48 horas
```

```
ORA-06512: en "PAU.TR_BORRA_AULA", línea 12
```

```
ORA-04088: error durante la ejecución del disparador 'PAU.TR_BORRA_AULA').
```

```
Rollback ejecutado para limpiar el test.
```

```
Procedimiento PL/SQL terminado correctamente.
```

13

14

script.sql

```
-- 5.1. DESPISTE
CREATE OR REPLACE PROCEDURE DESPISTE (
    P_DNI_ESTUDIANTE IN ASISTENCIA.DNI_ESTUDIANTE%TYPE,
    P_FECHAHORA IN ASISTENCIA.FECHAHORA%TYPE,
    P_NUEVA_SEDE IN ASISTENCIA.COD_SEDE%TYPE,
    P_NUEVO_AULA IN ASISTENCIA.COD_AULA%TYPE
) IS
    V_PRIMERA_HORA    DATE;
    V_AULA_DISPONIBLE AULA.CODIGO%TYPE;

    -- Cursor para buscar el RESTO de exámenes del alumno en el MISMO DIA de la fecha del sistema
    -- Excluimos el examen actual que se pasa como parametro y que reubicamos primero.
    CURSOR C_EXAMENES_DIA IS
        SELECT E.CODIGO_MATERIA AS COD_MATERIA, A.FECHAHORA, A.COD_SEDE, A.COD_AULA
        FROM ASISTENCIA A
        JOIN EXAMEN E ON A.COD_SEDE = E.COD_SEDE AND A.COD_AULA = E.COD_AULA AND A.FECHAHORA = E.FECHAHORA
        WHERE A.DNI_ESTUDIANTE = P_DNI_ESTUDIANTE
            AND TRUNC(A.FECHAHORA) = TRUNC(P_FECHAHORA) -- Pra buscar exámenes del mismo día
            AND A.FECHAHORA ≠ P_FECHAHORA;

    -- Excepciones personalizadas
    E_FUERA_DE_TIEMPO EXCEPTION;
    E_SIN_AULA_LIBRE EXCEPTION;
    PRAGMA EXCEPTION_INIT(E_FUERA_DE_TIEMPO, -20001);
    PRAGMA EXCEPTION_INIT(E_SIN_AULA_LIBRE, -20002);

BEGIN
    -- A. Comprobamos si el examen solicitado esta planificado entre SYSDATE y SYSDATE + 1 hora (1/24)
    SELECT MIN(FECHAHORA) INTO V_PRIMERA_HORA
    FROM ASISTENCIA
    WHERE DNI_ESTUDIANTE = P_DNI_ESTUDIANTE
        AND FECHAHORA ≥ SYSDATE;

    -- Comprobamos si falta menos de una hora para ese examen
    IF V_PRIMERA_HORA IS NULL OR V_PRIMERA_HORA NOT BETWEEN SYSDATE AND (SYSDATE + 1/24) THEN
        RAISE_APPLICATION_ERROR(-20001, 'Error: El procedimiento solo funciona si falta menos de una hora p
    END IF;

    -- B. Reubicacion del primer examen
    UPDATE ASISTENCIA
    SET COD_SEDE = P_NUEVA_SEDE,
        COD_AULA = P_NUEVO_AULA
    WHERE DNI_ESTUDIANTE = P_DNI_ESTUDIANTE
        AND FECHAHORA = P_FECHAHORA;
```

salida.sql

```
-- C. Reubicar el resto de exámenes del día
-- Recorremos todos los demás exámenes que tiene el alumno el día de hoy
FOR R_EX IN C_EXAMENES_DIA LOOP
    -- Buscamos un aula en la nueva sede que tenga examen de esa misma materia/hora
    -- y cuya ocupacion actual de alumnos asignados sea menor que su capacidad de examen
    BEGIN
        SELECT A.CODIGO
        INTO V_AULA_DISPONIBLE
        FROM AULA A
        JOIN EXAMEN E ON A.CODIGO = E.COD_AULA AND A.CODIGO_SEDE = E.COD_SEDE
        WHERE A.CODIGO_SEDE = P_NUEVA_SEDE
            AND E.FECHAHORA = R_EX.FECHAHORA
            AND E.CODIGO_MATERIA = R_EX.COD_MATERIA
            AND A.CAPACIDAD_EXAMEN > (
                -- Contamos cuantos estudiantes estan ya asignados a este aula/examen
                SELECT COUNT(*)
                FROM ASISTENCIA ASIS
                WHERE ASIS.COD_SEDE = E.COD_SEDE
                    AND ASIS.COD_AULA = E.COD_AULA
                    AND ASIS.FECHAHORA = E.FECHAHORA
            )
        AND ROWNUM = 1; -- Si hay varias aulas libres, seleccionamos la primera que encuentre

        -- Si para alguno de los exámenes del día no hay espacio en la nueva sede, abortamos todo
        EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20002, 'Error: No hay aula disponible en la sede de destino para el examen de las '
                || TO_CHAR(R_EX.FECHAHORA, 'HH24:MI') || ' de la materia ' || R_EX.COD_MATERIA);
    END;

    -- Si ha encontrado aula libre, modificamos la asistencia del alumno para ese examen del día
    UPDATE ASISTENCIA
    SET COD_SEDE = P_NUEVA_SEDE,
        COD_AULA = V_AULA_DISPONIBLE
    WHERE DNI_ESTUDIANTE = P_DNI_ESTUDIANTE
        AND FECHAHORA = R_EX.FECHAHORA;

    END LOOP;
    COMMIT;

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK; -- En caso de cualquier error, deshacemos los cambios
        RAISE;
END DESPISTE;
/
```

```
-- TEST DE VERIFICACIÓN: DESPISTE -----  
SET SERVEROUTPUT ON;  
PROMPT Probando Procedimiento DESPISTE (Control de reubicación)...  
BEGIN  
    -- Forzamos error de negocio de un estudiante sin asistencia prevista enviando SYSDATE  
    BEGIN  
        DESPISTE('00000000T', SYSDATE, 1, 1);  
        DBMS_OUTPUT.PUT_LINE('FALLO: Traspasó la comprobación de < 1 hora incorrectamente.');    EXCEPTION  
        WHEN OTHERS THEN  
            IF SQLCODE = -20001 THEN  
                DBMS_OUTPUT.PUT_LINE('ÉXITO: La barrera inicial del procedimiento funciona (' || SQLERRM || ').');            ELSE  
                DBMS_OUTPUT.PUT_LINE('ERROR SECUNDARIO: ' || SQLERRM);  
            END IF;  
    END;  
    ROLLBACK;  
    DBMS_OUTPUT.PUT_LINE('Rollback ejecutado para el test DESPISTE.');END;  
/
```

12
13
14

script.sql

```
-- 5.2. MIGRAR_CENTRO
CREATE OR REPLACE TRIGGER TR_MIGRAR_CENTRO
AFTER UPDATE OF CODIGO_SEDE ON CENTRO
FOR EACH ROW
DECLARE
    V_AULA_DISPONIBLE    AULA.CODIGO%TYPE;

    -- cursor que usa los valores new y old del centro modificado
    CURSOR C_EXAMENES_ALUMNOS IS
        SELECT A.DNI_ESTUDIANTE, A.FECHA_HORA, E.CODIGO_MATERIA, A.COD_AULA AS AULA_VIEJA
        FROM ASISTENCIA A
        JOIN ESTUDIANTE EST ON A.DNI_ESTUDIANTE = EST.DNI
        JOIN EXAMEN E ON A.COD_SEDE = E.COD_SEDE AND A.COD_AULA = E.COD_AULA AND A.FECHA_HORA = E.FECHA_HORA
        WHERE EST.CODIGO_CENTRO = :NEW.CODIGO
        AND A.COD_SEDE = :OLD.CODIGO_SEDE;

BEGIN
    -- procesamos en cascada la reubicacion de todos los alumnos implicados
    FOR EXA IN C_EXAMENES_ALUMNOS LOOP
        BEGIN
            -- buscamos aula en la nueva sede que le acaban de poner al centro
            SELECT A.CODIGO INTO V_AULA_DISPONIBLE
            FROM AULA A
            JOIN EXAMEN E ON A.CODIGO = E.COD_AULA AND A.CODIGO_SEDE = E.COD_SEDE
            WHERE A.CODIGO_SEDE = :NEW.CODIGO_SEDE
            AND E.FECHA_HORA = EXA.FECHA_HORA
            AND E.CODIGO_MATERIA = EXA.CODIGO_MATERIA
            AND A.CAPACIDAD_EXAMEN > (
                -- control de capacidad para asegurar el límite
                SELECT COUNT(*)
                FROM ASISTENCIA ASIS
                WHERE ASIS.COD_SEDE = E.COD_SEDE
                AND ASIS.COD_AULA = E.COD_AULA
                AND ASIS.FECHA_HORA = E.FECHA_HORA
            )
            AND ROWNUM = 1;
        END LOOP;
    END TR_MIGRAR_CENTRO;
```

salida.sql

```
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        -- cancelamos el update del centro si no caben todos
        RAISE_APPLICATION_ERROR(-20051, 'Error: no se puede asignar por falta de capacidad en la nueva sede');
END;
```

```
-- reubicamos al alumno cambiando la sede y el aula asignada
UPDATE ASISTENCIA
SET COD_SEDE = :NEW.CODIGO_SEDE,
    COD_AULA = V_AULA_DISPONIBLE
WHERE DNI_ESTUDIANTE = EXA.DNI_ESTUDIANTE
AND FECHA_HORA = EXA.FECHA_HORA
AND COD_SEDE = :OLD.CODIGO_SEDE
AND COD_AULA = EXA.AULA_VIEJA;
```

```
-- TEST DE VERIFICACIÓN: TR_MIGRAR_CENTRO -----
SET SERVEROUTPUT ON;
PROMPT Probando trigger TR_MIGRAR_CENTRO en cascada...
DECLARE
    v_cod_centro CENTRO.CODIGO%TYPE;
    v_nueva_sede SEDE.CODIGO%TYPE;
BEGIN
    SELECT CODIGO INTO v_cod_centro FROM CENTRO WHERE ROWNUM = 1;
    SELECT CODIGO INTO v_nueva_sede FROM SEDE WHERE CODIGO ≠ (SELECT NVL(CODIGO_SEDE, -1) FROM CENTRO WHERE CODIGO = v_cod_centro) AND ROWNUM = 1;

    UPDATE CENTRO SET CODIGO_SEDE = v_nueva_sede WHERE CODIGO = v_cod_centro;
    DBMS_OUTPUT.PUT_LINE('ÉXITO: Centro actualizado. El trigger de migración en cascada no reportó excepciones incontroladas.');
```

ROLLBACK;

DBMS_OUTPUT.PUT_LINE('Rollback completado de la migración ficticia.');

EXCEPTION WHEN OTHERS THEN ROLLBACK;

DBMS_OUTPUT.PUT_LINE('FALLO EN EL ENTORNO O TRIGGER: ' || SQLERRM);

END;

/

11

12

13

14

Probando trigger TR_MIGRAR_CENTRO en cascada...

ÉXITO: Centro actualizado. El trigger de migración en cascada no reportó excepciones incontroladas.

Rollback completado de la migración ficticia.

Procedimiento PL/SQL terminado correctamente.

script.sql

```
-- 1. Funcion VPD que inyecta un filtro "WHERE".
-- Si el usuario es un estudiante (su sesion empieza por EST_), la base de datos
-- anade una condicion para que solo pueda consultar su propio registro
CREATE OR REPLACE FUNCTION F_VPD_FILTRO_ESTUDIANTE (
    P_SCHEMA IN VARCHAR2,
    P_TABLE IN VARCHAR2
) RETURN VARCHAR2 IS
    V_USER VARCHAR2(100);
BEGIN
    -- Usuario de la sesion actual
    V_USER := SYS_CONTEXT('USERENV', 'SESSION_USER');

    -- Si es un estudiante, limitamos su acceso a su propio registro
    IF V_USER LIKE 'EST_%' THEN
        RETURN 'USUARIO_ORACLE = ''' || V_USER || ''';
    ELSE
        -- Si es el administrador (PAU), SYSTEM o el Servicio de Acceso, tienen acceso total a todas las filas
        RETURN NULL;
    END IF;
END;
/
```

10

11

12

13

14

salida.sql

```
-- 2. Registramos la politica VPD en la tabla estudiante utilizando el paquete DBMS_RLS
BEGIN
    DBMS_RLS.ADD_POLICY(
        object_schema => 'PAU',
        object_name => 'ESTUDIANTE',
        policy_name => 'POLITICA_ESTUDIANTES_VPD',
        function_schema => 'PAU',
        policy_function => 'F_VPD_FILTRO_ESTUDIANTE',
        statement_types => 'SELECT, UPDATE, DELETE',
        update_check => TRUE
    );
EXCEPTION
    WHEN OTHERS THEN
        NULL;
END;
/
```

```
-- TEST DE VERIFICACIÓN: F_VPD_FILTRO_ESTUDIANTE -----  
SET SERVEROUTPUT ON;  
PROMPT Probando política de acceso VPD en estudiantes...  
DECLARE  
    v_count NUMBER;  
BEGIN  
    -- Comprobamos visibilidad. Al ser lanzada por PAU se deben ver todos  
    SELECT COUNT(*) INTO v_count FROM ESTUDIANTE;  
    DBMS_OUTPUT.PUT_LINE('ÉXITO: Política activa evaluada. Visibilidad del perfil PAU: ' || v_count || ' filas.');
```

END;
/ -- Comprobar con otro usuario y se confirma que sale solo sus filas correspondientes

8

9

10

11

12

13

14

```
Probando política de acceso VPD en estudiantes...  
ÉXITO: Política activa evaluada. Visibilidad del perfil PAU/DBA: 8599 filas.  
  
Procedimiento PL/SQL terminado correctamente.
```

```
-- 1. Tabla de audit
DROP TABLE AUDIT_ASISTENCIA;
CREATE TABLE AUDIT_ASISTENCIA (
    ID_LOG            NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    FECHAHORA_AUDIT   DATE DEFAULT SYSDATE,
    USUARIO_REALIZO   VARCHAR2(100),
    OPERACION         VARCHAR2(20),
    DNI_ESTUDIANTE    VARCHAR2(20),
    COD_MATERIA       VARCHAR2(10),
    COD_SEDE          NUMBER,
    COD_AULA          NUMBER,
    ASISTE_ANTIGUO    VARCHAR2(10),
    ASISTE_NUEVO      VARCHAR2(10)
);

-- 2. Trigger que coge los cambios en asistencia
CREATE OR REPLACE TRIGGER TR_AUDIT_ASISTENCIA
AFTER UPDATE OR DELETE ON ASISTENCIA
FOR EACH ROW
DECLARE
    V_OP VARCHAR2(20);
BEGIN
    IF UPDATING THEN
        V_OP := 'MODIFICACION';
    ELSIF DELETING THEN
        V_OP := 'BORRADO';
    END IF;

    INSERT INTO AUDIT_ASISTENCIA (
        USUARIO_REALIZO, OPERACION, DNI_ESTUDIANTE, COD_MATERIA, COD_SEDE, COD_AULA, ASISTE_ANTIGUO, ASISTE_NUEVO
    ) VALUES (
        SYS_CONTEXT('USERENV', 'SESSION_USER'),
        V_OP,
        :OLD.DNI_ESTUDIANTE,
        :OLD.COD_MATERIA,
        :OLD.COD_SEDE,
        :OLD.COD_AULA,
        :OLD.ASISTE,
        CASE WHEN V_OP = 'MODIFICACION' THEN :NEW.ASISTE ELSE NULL END
    );
END;
```


script.sql

salida.sql

```

1
2
3
-- TEST DE VERIFICACIÓN: TR_AUDIT_ASISTENCIA -----
SET SERVEROUTPUT ON;
PROMPT Probando trigger de auditoría TR_AUDIT_ASISTENCIA...
DECLARE
    v_count      NUMBER;
    v_dni_est    VARCHAR2(9);
    v_dni_voc    VARCHAR2(9);
    v_mat        VARCHAR2(10);
    v_sede       NUMBER;
    v_aula       NUMBER;
    v_fecha      TIMESTAMP := SYSDATE;
BEGIN
    -- Capturamos datos DE LAS TABLAS tablas
    SELECT DNI INTO v_dni_est FROM ESTUDIANTE WHERE ROWNUM = 1;
    SELECT DNI INTO v_dni_voc FROM VOCAL WHERE ROWNUM = 1;
    SELECT CODIGO INTO v_mat FROM MATERIA WHERE ROWNUM = 1;
    SELECT CODIGO_SEDE, CODIGO INTO v_sede, v_aula FROM AULA WHERE ROWNUM = 1;

    -- 1. Insertamos primero el examen para cumplir la restricción de clave ajena compuesta
    INSERT INTO EXAMEN (FECHAHORA, CODIGO_MATERIA, COD_AULA, COD_SEDE, DNI_RESPONSABLE)
    VALUES (v_fecha, v_mat, v_aula, v_sede, v_dni_voc);

    -- 2. Insertamos en asistencia
    INSERT INTO ASISTENCIA (DNI_ESTUDIANTE, FECHAHORA, COD_MATERIA, COD_SEDE, COD_AULA, ASISTE)
    VALUES (v_dni_est, v_fecha, v_mat, v_sede, v_aula, 'N');

```

```

-- 3. Actualizamos para activar el trigger de auditoria
UPDATE ASISTENCIA SET ASISTE = 'S'
WHERE DNI_ESTUDIANTE = v_dni_est AND FECHAHORA = v_fecha;

-- 4. Consultamos la tabla de logs para comprobar el funcionamiento
SELECT COUNT(*) INTO v_count FROM AUDIT_ASISTENCIA
WHERE DNI_ESTUDIANTE = v_dni_est AND OPERACION = 'MODIFICACION';

IF v_count > 0 THEN
    DBMS_OUTPUT.PUT_LINE('ÉXITO: El registro de alteración se alojó en la tabla AUDIT_ASISTENCIA.');
```

```

ELSE
    DBMS_OUTPUT.PUT_LINE('FALLO: El trigger no generó rastreo auditor.');
```

```

END IF;

-- 5. Limpieza
ROLLBACK;
DBMS_OUTPUT.PUT_LINE('Rollback realizado. Todo devuelto a su estado original de forma limpia.');
```

```

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('FALLO CONTROLADO EN EL TEST: ' || SQLERRM);
END;
/

```

```

Probando trigger de auditoría TR_AUDIT_ASISTENCIA...
ÉXITO: El registro de alteración se alojó en la tabla AUDIT_ASISTENCIA.
Rollback realizado. Todo devuelto a su estado original de forma limpia.

Procedimiento PL/SQL terminado correctamente.

```